

Assessment: Event Planning Application

Objective

Build a full-stack event planning application that allows users to create, manage, and browse events with secure authentication and a responsive UI. The project should demonstrate practical full-stack development skills in a real-world scenario.

Core Features

Events

- Create new events (title, description, date/time, location).
- Edit existing events.
- Delete events.
- View a list of:
 - Upcoming events.
 - Past events.
- View details of a single event.

Tags & Categories

- Assign multiple tags to an event (e.g., "Birthday", "Conference", "Workshop").
- Filter events by:
 - Tags.
 - Event type (public/private).

Authentication & Authorization

- User signup and login.
- Secure authentication using JWT.
- Authorization so that only the event creator can edit or delete their events.

Technical Requirements

Tech Stack

- **Frontend:** React with TypeScript
- **Backend:** Node.js (Express with TypeScript).
- **Database:** MySQL (or any relational DB).
- **Preferred Query Builder:** Knex.js or any other query builders (**No ORMs i.e. TypeORM, Prisma, etc.**)
- **Preferred Local DB Service:** Docker Compose to run the database in a Docker container (optional).

Engineering Expectations

- Clean, responsive, and user-friendly UI.
- Validations for data integrity on both frontend and backend.
- Clear and user-friendly error messages for validation and server errors.
- Graceful error handling in both frontend and backend.
- Secure authentication implementation.
- RESTful API conventions.
- Server-side pagination and filtering for event listings.
- Normalized database schema design.

Optional Features (Preferred & Additional)

These features are not required but will enhance the application's functionality and demonstrate advanced skills.

- **Database & Backend**
 - Use Knex.js for migrations and queries.
 - Backend logging.
 - Migration scripts for database schema changes.
- **Frontend Enhancements**
 - Reusable UI components (forms, modals, buttons, etc.).
- **Testing & Documentation**
 - Unit tests (additional coverage welcome).
 - API documentation using Swagger/OpenAPI.
- **Advanced Authentication**
 - Refresh token implementation.
 - Two-Factor Authentication (2FA).
 - Email verification.

- **Event Management Enhancements**
 - **Search:** Find events by title, description, or location.
 - **Sorting:** Sort events by date, popularity, or creation time.
 - **RSVP System:** Users can RSVP as Yes/No/Maybe.

Documentation Requirements

Include a **README** with:

1. **Engineering Decisions** – Explain key architectural and technology choices.
2. **Setup Instructions** – Steps to run the project locally.
3. **Assumptions** – Any assumptions made during development.

Delivery Notes

- Implement **core features** first.
- Add **optional features** if time allows.
- Frameworks and libraries are allowed, but avoid full pre-built solutions.
- Be able to justify all engineering decisions.

Bonus Section

You may write your answer as a **SQL SELECT query**, **pseudocode**, or **plain English steps**. There is no single right syntax — we are evaluating your **thought process and logic**.

Context

Our application tracks two things for every employee — their **designation history** and their **project allocation history**. Each time something changes, a new row is inserted into the relevant table.

Table 1: **emp_designation_log**

Column	Type	Description
<code>txn_id</code>	VARCHAR	Unique transaction ID

emp_id	VARCHAR	Employee identifier
emp_name	VARCHAR	Employee full name
designation	VARCHAR	Designation at this point in time
effective_date	DATE	Date this designation became effective

Sample Data:

txn_id	emp_id	emp_name	designation	effective_date
T001	EMP001	Alice Johnson	Associate Developer	2024-02-01
T002	EMP001	Alice Johnson	Mid Developer	2024-02-05
T003	EMP001	Alice Johnson	Senior Developer	2024-02-10
T004	EMP002	Bob Martinez	Mid Developer	2024-05-02
T005	EMP002	Bob Martinez	Senior Developer	2024-07-15
T006	EMP002	Bob Martinez	Mid Developer	2024-09-20
T007	EMP003	Carol Smith	Mid Developer	2024-08-06
T008	EMP003	Carol Smith	Mid Developer	2024-08-06
T009	EMP004	David Lee	Associate Developer	2024-01-10
T010	EMP004	David Lee	Associate Developer	2024-04-10
T011	EMP004	David Lee	Mid Developer	2024-09-10
T012	EMP005	Eva Chen	Senior Developer	2024-06-15
T013	EMP005	Eva Chen	Mid Developer	2024-03-01
T014	EMP005	Eva Chen	Senior Developer	2024-11-20
T015	EMP006	Frank Patel	Associate Developer	2024-01-01

T016	EMP006	Frank Patel	Mid Developer	2024-05-10
T017	EMP006	Frank Patel	Mid Developer	2024-05-10
T018	EMP007	Grace Kim	Senior Developer	2023-03-03
T019	EMP007	Grace Kim	Resigned	2023-06-30
T020	EMP007	Grace Kim	Associate Developer	2024-01-15
T021	EMP007	Grace Kim	Mid Developer	2024-07-15
T022	EMP008	Henry Walsh	Associate Developer	2024-06-01
T023	EMP008	Henry Walsh	Mid Developer	2024-06-01
T024	EMP009	Irene Novak	Senior Developer	2024-09-01

Table 2: emp_allocation_log

Column	Type	Description
allocation_id	VARCHAR	Unique allocation ID
emp_id	VARCHAR	Employee identifier
project_name	VARCHAR	Project the employee was allocated to
allocated_role	VARCHAR	Role played in the project
allocation_start	DATE	Date allocation began
allocation_end	DATE	Date allocation ended (NULL = currently active)

Sample Data:

allocation_id	emp_id	project_name	allocated_role	allocation_start	allocation_end
---------------	--------	--------------	----------------	------------------	----------------

A001	EMP001	Project Alpha	Developer	2024-02-03	2024-04-30
A002	EMP001	Project Beta	Tech Lead	2024-05-01	2024-09-30
A003	EMP002	Project Alpha	Developer	2024-05-10	2024-08-31
A004	EMP002	Project Gamma	Senior Contributor	2024-09-01	NULL
A005	EMP003	Project Beta	Developer	2024-08-06	2024-12-31
A006	EMP004	Project Delta	Developer	2024-02-01	2024-10-31
A007	EMP005	Project Alpha	Senior Contributor	2024-04-01	2024-07-31
A008	EMP005	Project Gamma	Tech Lead	2024-08-01	NULL
A009	EMP006	Project Delta	Developer	2024-03-01	2024-06-30
A010	EMP007	Project Beta	Developer	2024-02-01	2024-06-30
A011	EMP008	Project Alpha	Developer	2024-07-01	NULL
A012	EMP009	Project Gamma	Senior Contributor	2024-10-01	NULL

Key relationship: Both tables share `emp_id`. There is no direct foreign key between them — the designation that was active during an allocation must be **derived from the dates**.

Questions

Q1

The product team wants to display a **profile badge** for each employee showing their current designation.

Write a query to return the **current designation** of every employee — defined as the designation from their most recent `effective_date`.

None

```
emp_id | emp_name | current_designation
```

Q2

A new screen in the application shows each employee's **designation timeline** — a side-by-side view of where they came from and where they are going.

Write a query that returns, for every row in the table:

None

```
emp_id | effective_date | previous_designation | designation |  
next_designation
```

Where `previous_designation` is the designation held just before this row (for the same employee), and `next_designation` is the one that comes after. Return `NULL` where there is no previous or next.

Q4 (8 marks)

The HR team has raised a request for a new report:

"For each project allocation, we want to know what designation the employee held at the time they were allocated to that project."

For example — if Alice joined **Project Alpha** on `2024-02-03`, and her designation on that date was `Associate Developer` (she became `Mid Developer` only on `2024-02-05`), the report should show `Associate Developer` against that allocation.

Write a query using **both tables** to produce the following output:

None

```
allocation_id | emp_id | emp_name | project_name | allocated_role |  
allocation_start | designation_at_allocation
```

Things to think about before writing your query:

- An employee may have had multiple designations over time. Only one was active on `allocation_start`.
- The designation table does not store an end date — you need to figure out the "active" designation on a given date purely from the history of rows.
- What happens if an employee has no designation record before their `allocation_start`?

You may write this as SQL, pseudocode, or explain your join strategy step by step. Partial credit is awarded for correct reasoning even if the final syntax is incomplete.